Open in app ↗

Medium Search Write

Get unlimited access to the best of Medium for less than $1/week. Become a member

Kubernetes observability with OpenTelemetry and Jaeger

Yuri Fenyuk Follow 8 min read · Dec 10, 2024

6 1

Having a Kubernetes cluster means hosting many computing resources inside. This amount quickly goes up, no matter which architecture solutions have. This is particularly the case with microservices. While a developer can maintain and troubleshoot any part of the solution in a playground, troubleshooting a composed solution deployed to Kubernetes is a tricky thing. Localizing the problem is a doable task with implemented ***Observability*** *in distributed solutions. This article shows a practical approach to setting up monitoring solution and benefiting from it,* ***even when it is hard/impossible to modify your microservices****.*

OpenTelemetry is a well-established Observability framework. It should be used to generate, gather, collect, and export your solution's telemetry data (metrics, traces, and logs). There are two approaches to extracting telemetry from your running microservices, known as Instrumentation:

- **Code-based** (or manual): when Developers modify their code to import OpenTelemetry libraries, add lines of code to produce custom telemetry and DevOps rebuild and redeploy microservices.
- **Zero-code** (or auto-instrumentation): when DevOps enriches microservice deployments in Kubernetes with observability components, *without changing source code at all.*

Zero-code (or auto-instrumentation) is a particularly promising approach, because it might be time-consuming and difficult to change source code. This approach can and will help localize performance issues in already developed solutions.

Jaeger is one of the possible Observability backends, which should import telemetry data from OpenTelemetry, store it and provide search and visualization(UI) capability. Jaeger is quite mature, as confirmed by its graduated status on CNCF. Even more interesting, is that after a few years of evolving in many releases of the first version, the new major Jeager v.2.1 (as of the end of 2024) is announced to go live soon.

Altogether, the plan for the article is:

- use kind (Kubernetes-IN-Docker) to run Kubernetes cluster locally;
- install Jaeger Operator in a simpler version to consume OpenTelemetry data;
- install OpenTelemetry Operator and configure it to do Zero-code instrumentation (which is individual and tricky as custom services are implemented with different Programming languages);
- install the old-chap Vote microservice sample application as a Kubernetes deployment and configure its vote-part (Python-written) and result-part(Node.js-written) to be *Zero-code Instrumented* with OpenTelemetry. This code was created years ago when nobody heard about OpenTelemetry;
- install my hand-crafted backend service (Golang-written), which I used in my recent articles;

- run tests on two of the above services, and see telemetry records in Jaeger UI;

It is important to stress, that I use both test Deployments *without any change in code or re-build.* And still receive some traces, which might give a hint of "how quick/reliable/healthy etc." arbitrary custom code behaves itself in Kubernetes. If this is not enough, Code-based Instrumentation is to the rescue.

First thing comes first. Let's create Kubernetes cluster, which also points *kubectl* to newly created cluster:

```
(base) y.fenyuk@UAR-M-V46VL40JHW articles_playground % kind create cluster
Creating cluster "kind" ...
 ✓ Ensuring node image (kindest/node:v1.31.2) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-kind"
You can now use your cluster with:

kubectl cluster-info --context kind-kind

Not sure what to do next? 😅  Check out https://kind.sigs.k8s.io/docs/user/quick-start/
```

kind create cluster

Cert Manager is required, so needs to be installed.

The next step is to install Jaeget Operator:

```
kubectl create namespace observability
kubectl create -f https://github.com/jaegertracing/jaeger-operator/releases/download/v1.4
```

install-kaeger.sh hosted with ❤ by GitHub view raw

install Jaeger

Going the simplest way to deploy "all-in-one" Jaeger instance:

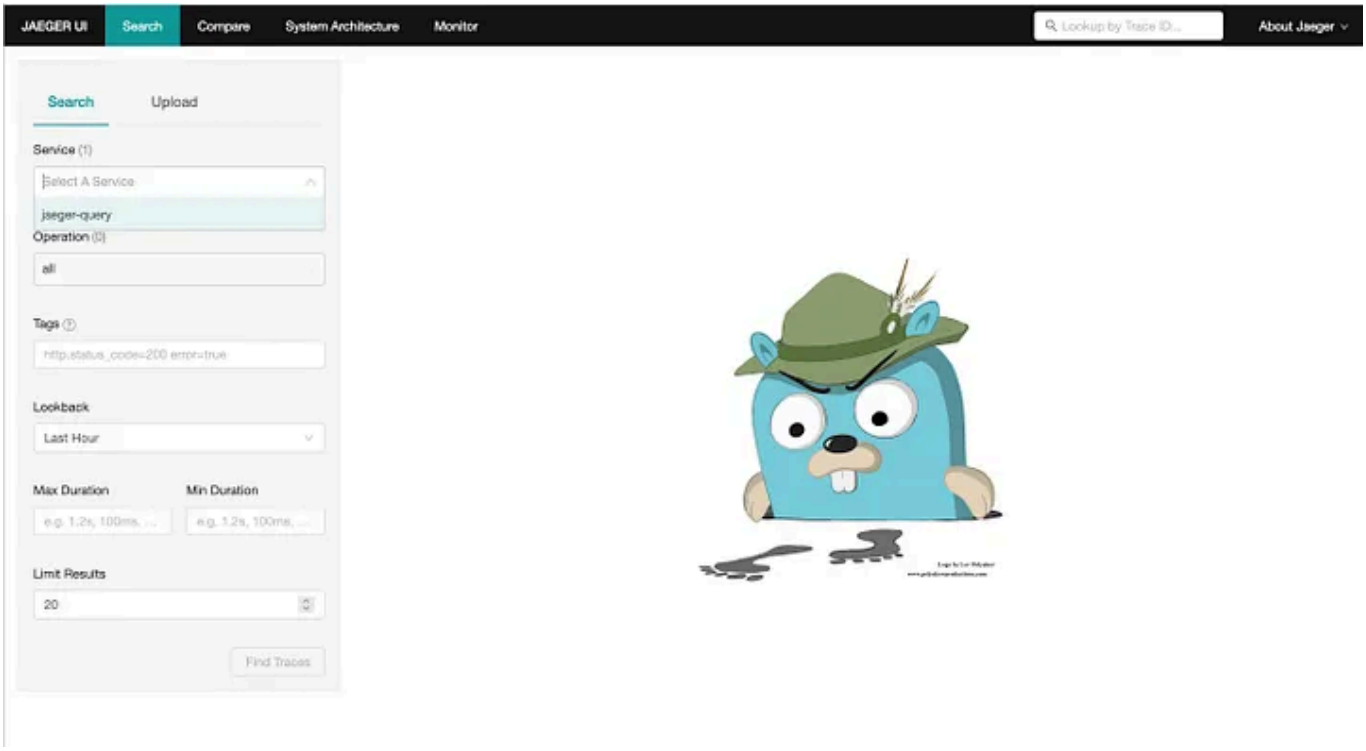
```yaml
apiVersion: jaegertracing.io/v1
kind: Jaeger
metadata:
   name: jaeger
   namespace: observability
```

jeager-instance.yaml hosted with ❤ by GitHub view raw

jeager-instance.yaml

with:

This deployment also creates Kubernetes service (in this case with name *jaeger-query)* with UI, which is empty at the moment (in fact, only has itself in the service dropdown):



empty Jaeger UI

Time to install and configure OpenTelemetry.

The first part is to install OperTelemetry Operator. The installation is pretty straightforward unless you plan Auto-instrument (or Zero-code, which is a synonym here) Golang applications. My guess, is that the biggest part of Kubernetes is written with Golang, which is why this option is disabled on normal installation.

Please follow the accepted solution in *"How to pass "enable-go-instrumentation" flag to OpenTelemetry Operator?"* question and copy whole deployment YAML file locally to add extra flag `--enable-go-instrumentation=true`.

This should conclude OpenTelemetry CRD installation and two CRs (custom resources) should be created. It is important to read the documentation details and google in case of errors.

OpenTelemetryCollector: below is the instruction for Collector, which takes every telemetry that has been captured by agents and sends it to Jaeger's Collector:

configuring Collector

#10..#11: subscribes to receive available at GRPC endpoint metrics. It is interesting, that only Node.js Auto-instrumentation by default uses GRPC;

#12..#13: same for HTTP and protobuf. This is for Python and Golang Auto-instrumentation;

#24..#27: output metrics from Collector to Jaeger, which has been created in observability namespace;

#28: also output metrics from Collector to standard console.

Instrumentation: with this CR, instructing OpenTelemetry what to instrument and to which Collector to send captured telemetry:

configure Instrumentation

#6..#7: by default, send all telemetry from clients' Pod to the Collector in the same namespace. Documentation states (and it also mentioned in previous YAML) that 4317 is GRPC port;

#14..#22: customize Auto-instrumentation for Python applications and route telemetry data to port 4318;

#23..#26: customize Auto-instrumentation for Golang applications.

Both YAML files need to be deployed with *kubectl* and, hopefully, will not report errors. To check deployments:

which on my laptop shows:

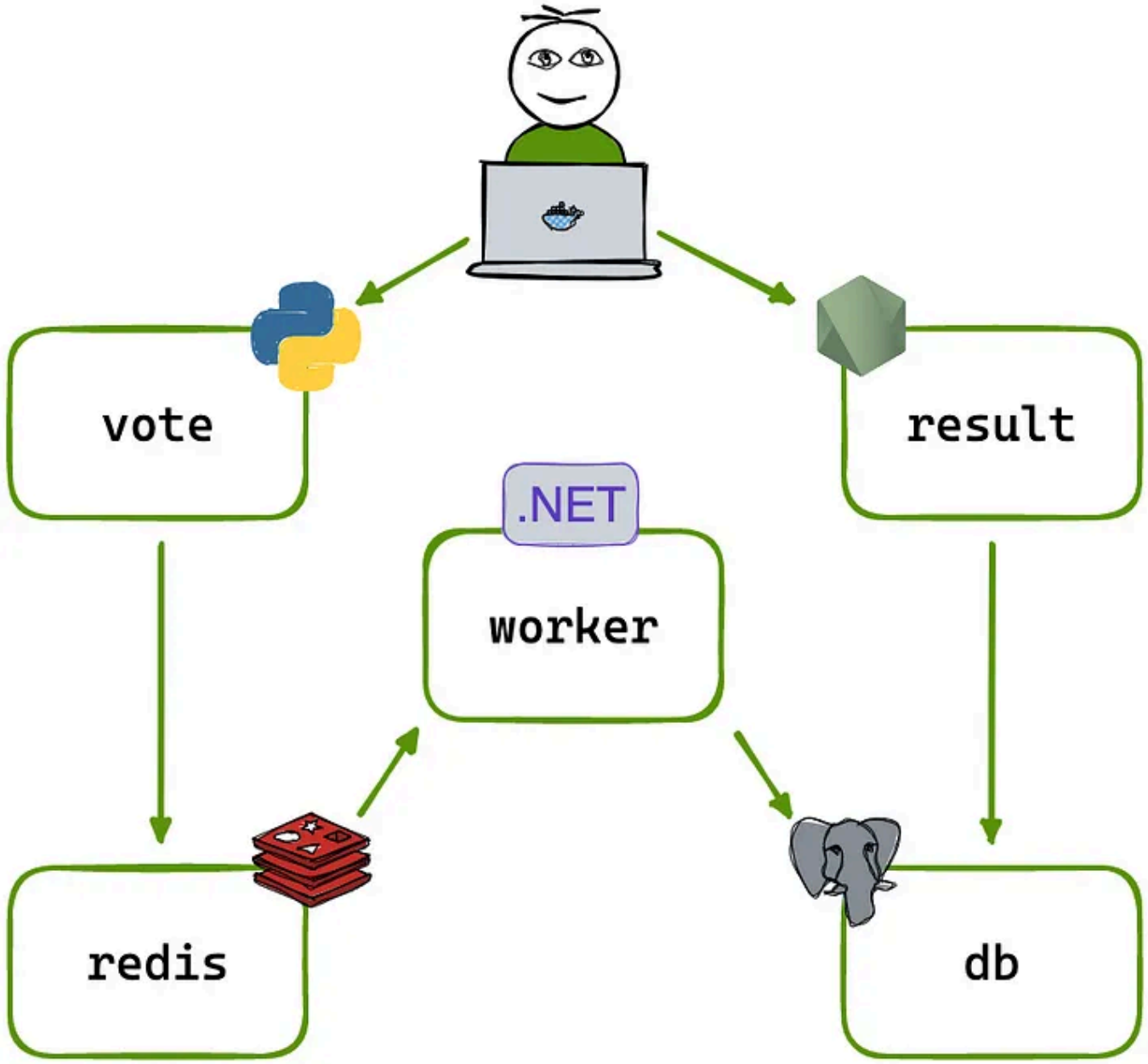
```
(base) y.fenyuk@UAR-M-V46VL40JHW articles_playground % kubectl get deploy otelcol-collector

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
otelcol-collector   1/1     1            1           4h32m
(base) y.fenyuk@UAR-M-V46VL40JHW articles_playground % kubectl get instrumentation
NAME                   AGE   ENDPOINT                          SAMPLER                    SAMPLER ARG
demo-instrumentation   12m   http://otelcol-collector:4317   parentbased_traceidratio   1
```

both Custom Resources are run

I know, this sequence is way too long, but this was required because of customizations. I made it work only after reaching several dead-ends and restarting the whole process from the beginning. And you (lucky people:))have it here described.

Finally, it is time to deploy test applications. Here is microservice architecture of voting test application:



Per OpenTelemetry documentation, a few lines in Pod's annotation are required.

Please download example-voting-app locally. Kubernetes definitions in folder k8s-specifications, needs a minimal tweak.

Service 'vote' is written in Python, so it Pod annotation needs to have one new line:

vote-deployment.yaml

#17: instructing OpenTelemetry to Auto-instrument all Pods from this Deployment.

All other lines are left unchanged!
Please, notice, that there is no need to change anything in Docker image! We continue to use existing *dockersamples/examplevotingapp_vote. T*hat is the real power of **Zero-code instrumentation**.

Service ‘result’ is written in Node.js and the very same job needs to be done with it Kubernetes deployment definition:

result-deployment.yaml

#17: new line to inject OperTelemetry for Node.js agent.

All other lines are left unchanged!

Follow microservice deployment section, let deploy it slightly modified version with `kubectl create -f k8s.specifications/` and in half of minute, all Pods this example consists of should be running:

Pods 6 items Namespace: default Search Pods...

Name	Namespace	Containers	Restarts	Controlled ...	Node	QoS	Age	Status
db-5b8f566868-2xj45	default		0	ReplicaSet	kind-control-plan	BestEffort	5m45s	Running
otelcol-collector-5d795949f9-2l...	default		0	ReplicaSet	kind-control-plan	BestEffort	5h35m	Running
redis-5b65678587-nmzzf	default		0	ReplicaSet	kind-control-plan	BestEffort	5m45s	Running
result-85c695b7df-knjp2	default		0	ReplicaSet	kind-control-plan	Burstable	5m45s	Running
vote-75b4754c4b-9pslr	default		0	ReplicaSet	kind-control-plan	Burstable	5m45s	Running
worker-765456c688-nfwjg	default		0	ReplicaSet	kind-control-plan	BestEffort	5m45s	Running

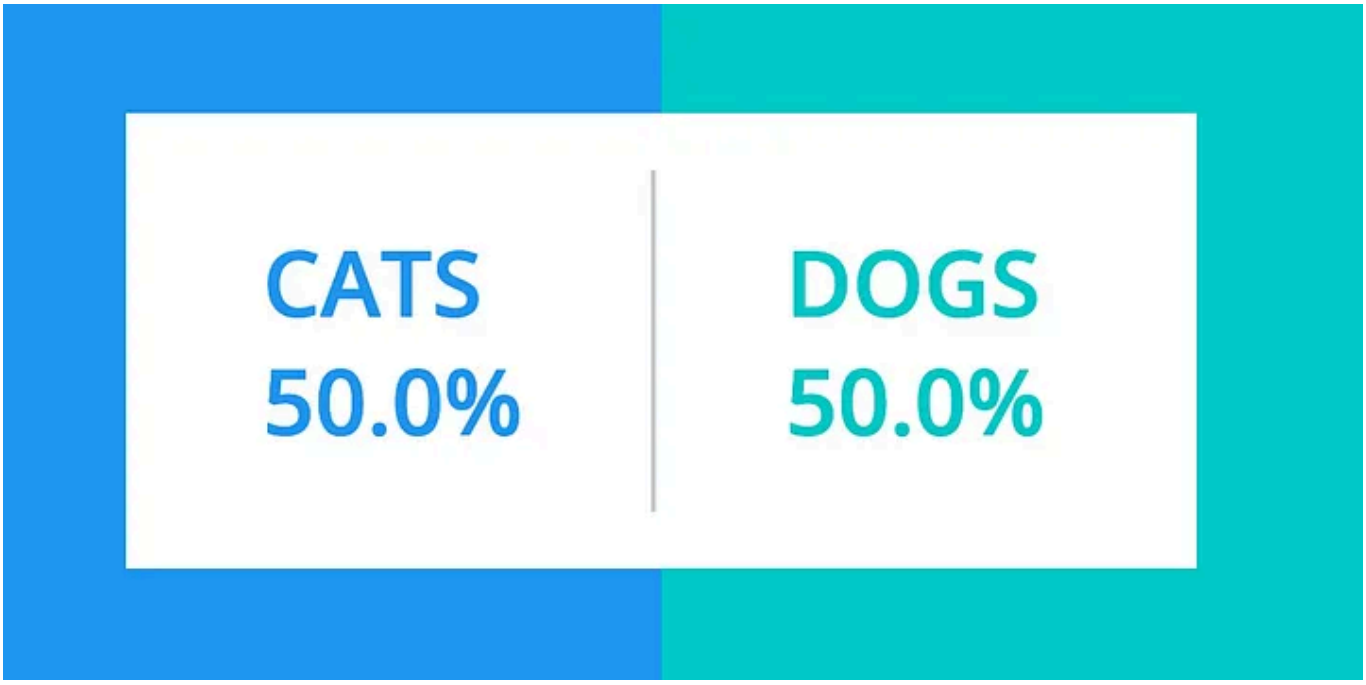
Pods in Lens GUI

Screenshot has in column *Containers* two icons (green and grey) for *vote* and *result* Pods. Thease are traces of init containers of OpenTelemetry, which instrumented main Pod's container and exited. From now, some activity in Pod is intercepted and sent to OpenTelemetryCollector, which, in turns, sends to Jager's storage!

Let's forward port for service 'vote' and click one for cats and for dogs:

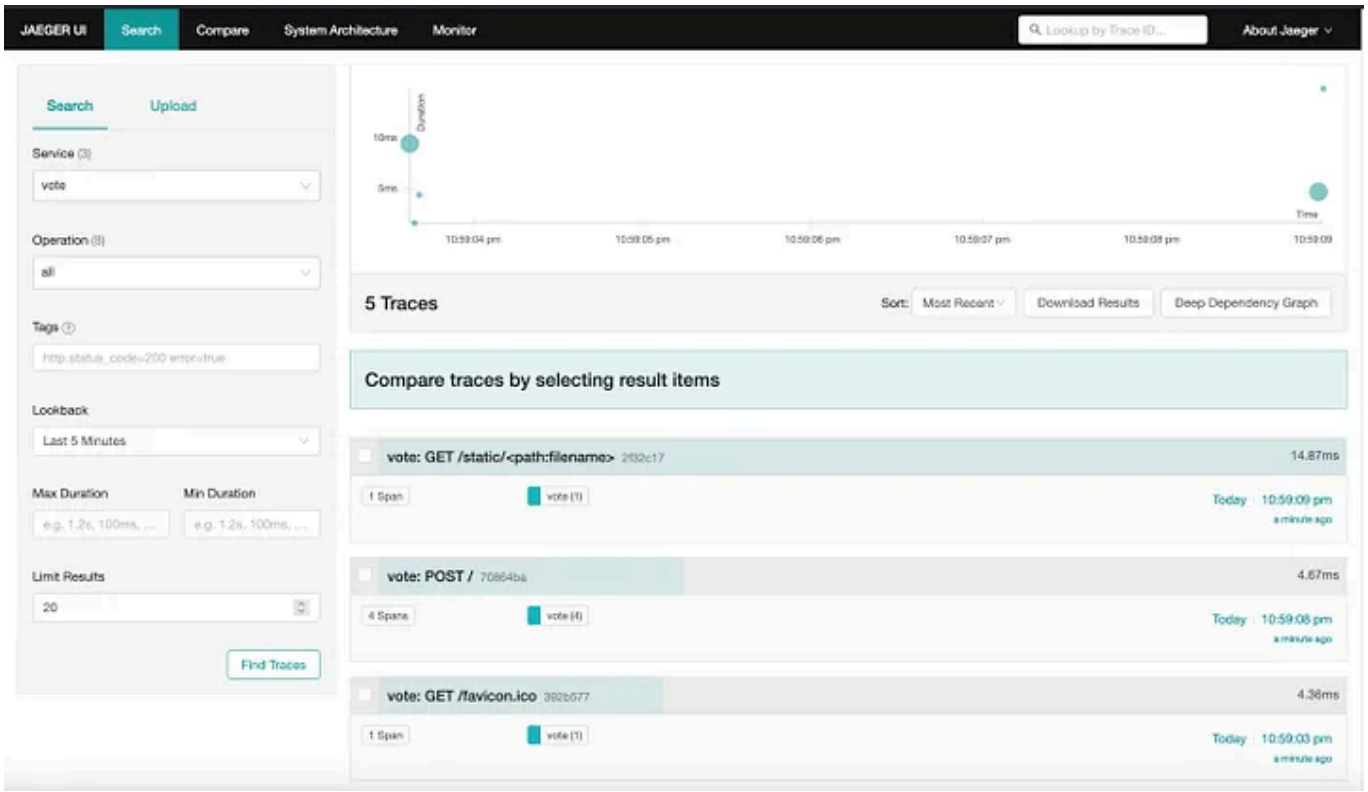
‘vote’ service UI

and query for results in ‘result’ service:



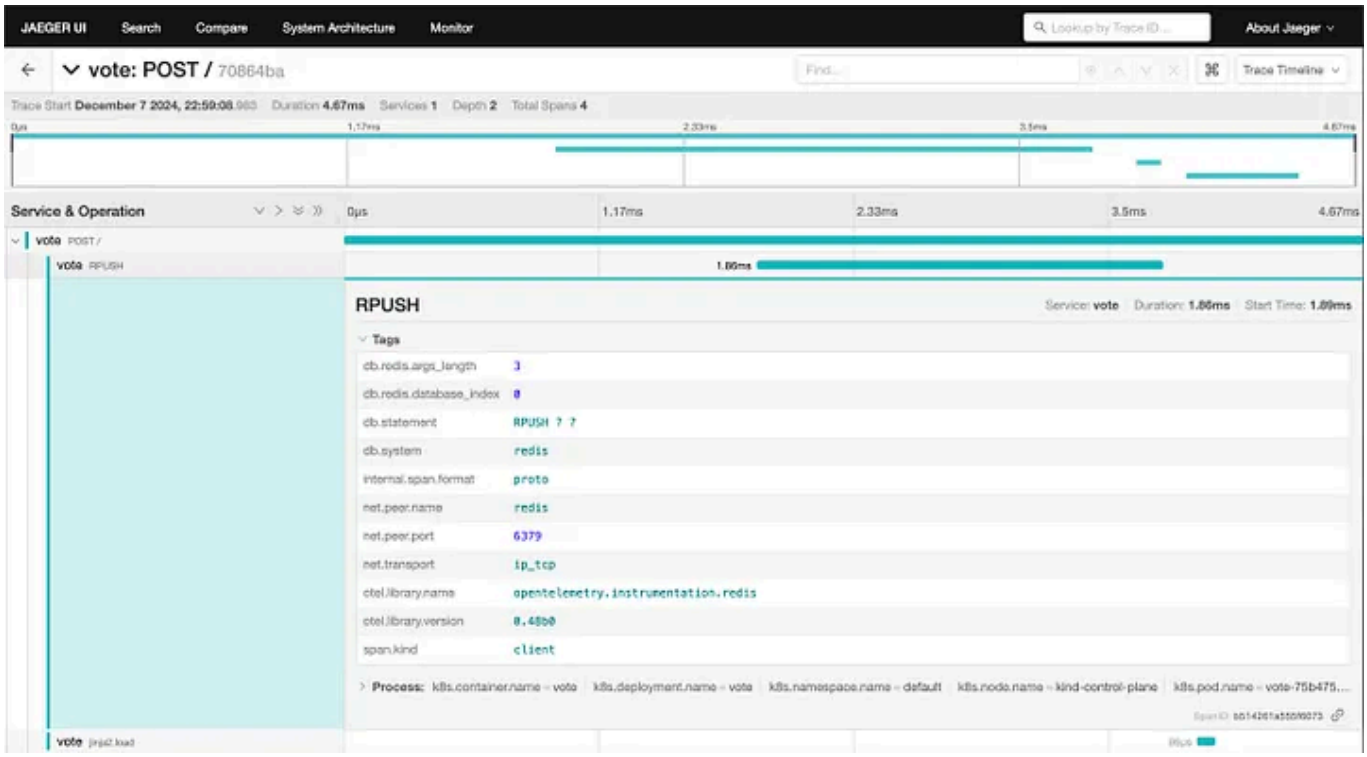
‘result’ service UI

When we go back to Jaeger UI and refresh, we will see three services (not one as it used to be). Select ‘vote’ service to see recent activity:



Jaeger for ‘vote’ service

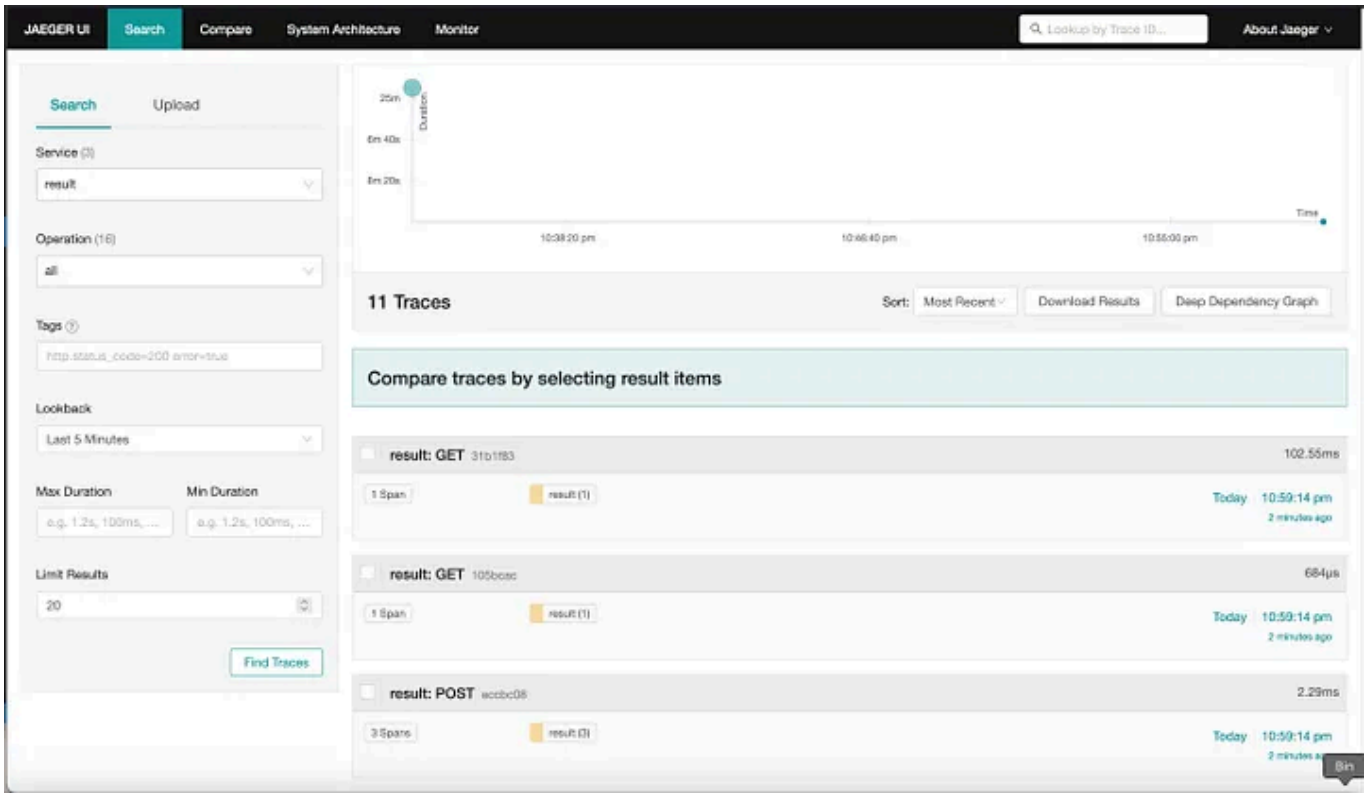
And by clicking POST method, following details are visible:



POST span

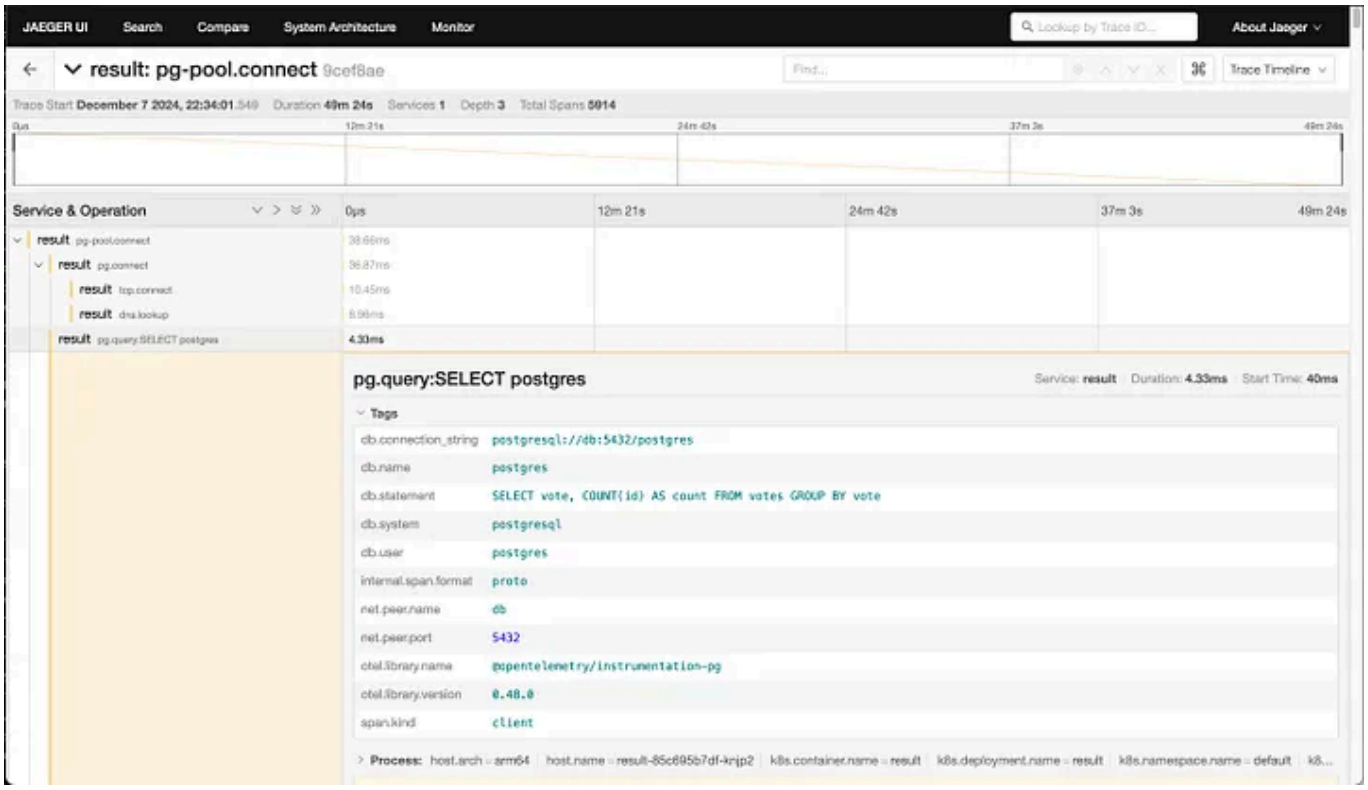
which shows that total method call time is 3.78ms and half this tiny time is Redis RPUSH command. Please recall that these details are extracted from Auto-instrumented Pod, whose code is untouched. For sure, Code-based instrumentation brings more business value on the table, although it requires more changes.

Let’s select ‘result’ service in Jaeger UI and see it recent telemetry:



Jager for ‘result’ service

The following span is particularly interesting:



‘results’ taking to DB

Auto-instrumentation in this case, not only managed to categorize the call as Postgres request, but even showed executed SQL(no surprise, ‘result’ service wants to know grouped voting results).
Again, no containerized Node.js application were even touched! It was instrumented from outside by injection init-container in the Pod. And it can be easily de-instrumented, with one line in *result-deployment.yaml.*

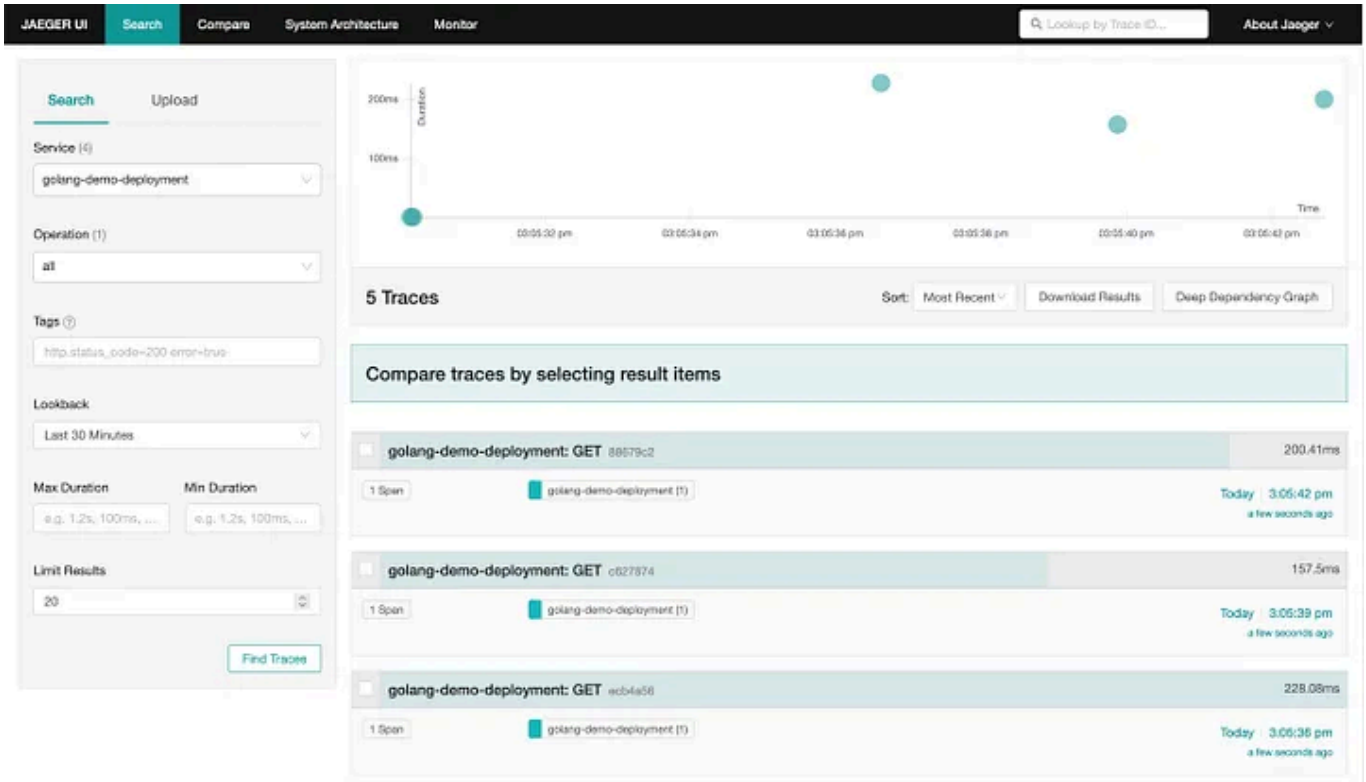
Having reached that much with OpenTelemetry and Zero-code instrumentation for Python and Node.js custom services, the final step is to cover Golang written service. It is simple due to the trick with `--enable-go-instrumentation=true` that was described before. The plan is to use Golang written demo-service-hashing I used in the previous article (Kubernetes definitions are here).
Similarly, OpenTelemetry specific annotations need to be added to Pod’s declaration. Below are changes:

#7: this time instrumenting Golang;

#8: Specify startup executable (in my case, it is named *'main'* and located in folder *'app'*);

#13..#15: per OpenTelemetry documentation, Auto-instrumentaion requires Container to run in privileged mode.

Straightforward installation with *kubectl*, port forward for its Service and quick endpoint test in the browser, should push data down the OpenTelemetry flow to Jaeger and make it's visible UI:



Jaeger UI with Golang written spans

One more service has appeared in the dropdown in Jaeger UI. There are 3 GET requests visible in UI with simple, not worth looking at implementation.

Gitlab repo folder with mentioned in article source files.

This article just scratches the surface of OpenTelemetry, but still presents a working approach to Zero-code (Auto-instrumentation) as a quick way to observe and troubleshoot microservice architecture applications inside Kubernetes. OpenTelemtry's extensible and production scale design makes it possible to export

caught telemetry into different open-sourced backends, like Jaeger or Prometheus, as well as other commercial products, or hand-craft bridge to something else.

Written by Yuri Fenyuk

167 followers · 23 following

Experienced software developer

Follow

Responses (1)

Anand Ray

What are your thoughts?

S

Sunilsureshrajesh

May 5

Thanks for the blog. I am unable to see result service in jaeger UI. what could have gone wrong.

1 1 reply Reply

More from Yuri Fenyuk

Yuri Fenyuk

Kubernetes observability with OpenTelemetry and Grafana

In my previous article, Kubernetes observability with OpenTelemetry and Jaege...

Apr 29 25

Yuri Fenyuk

Helm for Kubernetes. Handling secrets with SOPS

The first Kubernetes related article I wrote contains a practical introduction to Helm...

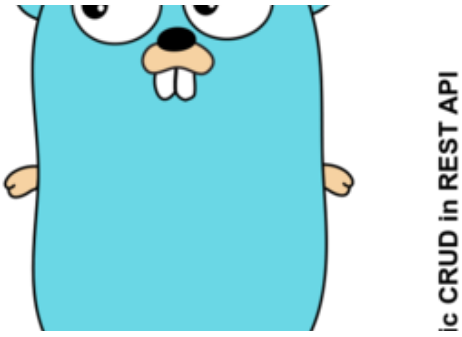
May 23, 2022 71

Yuri Fenyuk

Building a Kubernetes Controller with Kubebuilder

In this chapter, I continue exploring the powerful world of Kubernetes customization...

May 6 1



Yuri Fenyuk

Golang. CRUD in REST API in a generic way.

Building REST API and implementing CRUD operations are among the most common...

Apr 17, 2023 59

See all from Yuri Fenyuk

Recommended from Medium

Ömer Ceylan

Distributed Tracing in Kafka Messages with OpenTelemetry in...

In modern microservice architectures, tracking events across services...

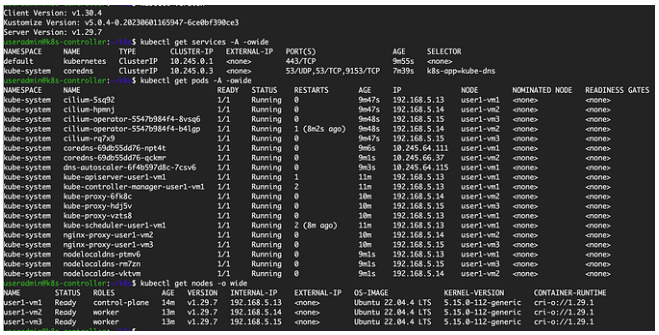
Feb 26

Yogender Pal

Microservices Monitoring with Grafana & Prometheus in...

Deploy K8S cluster, Deploy Prometheus & Grafana, Deploy Microservices, Monitor...

Mar 26



In T3CH by DaeGon Kim

Kubespray Deployment with Custom Configurations

Kubernetes Versions, Container Runtime, CNI and Subnets

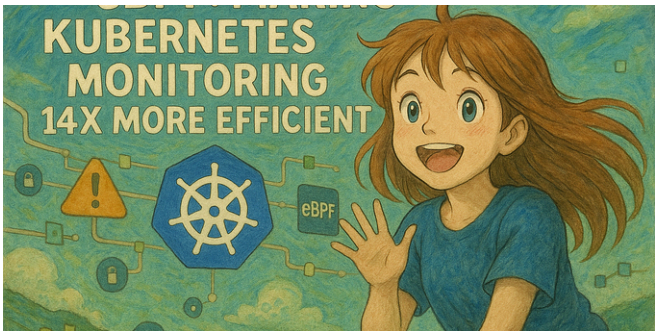
Dec 3, 2024 54

Codepinion

Why the Lack of Zig Frameworks Could Stall Its Growth

Zig is one of the most exciting system programming languages of the decade. It's...

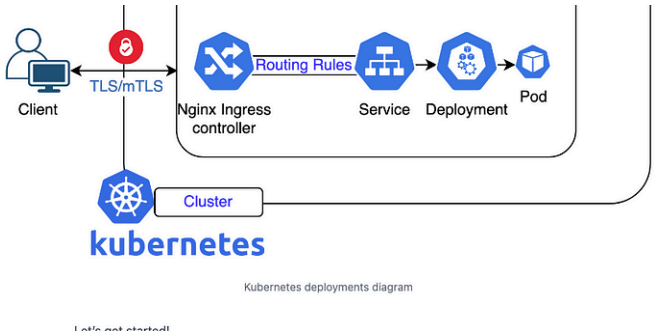
May 3 1



In devdotcom by Baivab Mukhopadhyay

eBPF: Making Kubernetes Monitoring 14X More Efficient

Ever got a 2 a.m. alert that your Kubernetes app is down? You're digging through logs,...



Aidudo Nicholas

Deploying NGINX as an Ingress Controller in Kubernetes: A...

Using NGINX as an Ingress Controller in Kubernetes: A Step-by-Step guideIn modern...

May 5 83 2 Nov 22, 2024 2

See more recommendations